INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR, ADYAR - CHENNAI

GROWTHAI: A SKILL-GAP DRIVEN PERSONAL GROWTH AND TIME-MANAGEMENT ASSISTANT

ABDUL KADER MAAHIR S – 24619202

P. KISHORE – 24619262

MINI PROJECT REPORT

Submitted to the

SCHOOL OF ENGINEERING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

in Partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY CSE

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI – 600 117

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report "**GROWTHAI: A SKILL-GAP DRIVEN PERSONAL GROWTH AND TIME-MANAGEMENT ASSISTANT**", is the Bonafide work of **ABDUL KADER MAAHIR S - 24619202** and **P. KISHORE - 24619262** carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate

SIGNATURE OF THE HOD

Dr. K. Ulaga Priya M.E., Ph.D
Associate Professor & Head
Of Department AI&ML
School of Engineering
VISTAS
Chennai

SIGNATURE OF THE SUPERVIOR

Ms. V. P. ANITHA
Assistant Professor
Department of AI&ML
School of Engineering
VISTAS
Chennai

Submitted for the Project Work and Viva - Voce examination held on……
at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support I received from the positive minds around me in making this venture a successful one. The infrastructural support with all kinds of lab facilities have been a motivating factor in this completion of project work, all because of FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH., with great pleasure that I take this opportunity to thank him. I am grateful to our **HEAD OF THE DEPARTMENT, Dr. K. Ulaga Priya,** for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day. I extend our warmest thanks to my **INTERNAL GUIDE, Ms. V. P. ANITHA Assistant Professor/AI&ML,** who has patiently guided and provided us with invaluable advice and help. I would like to express my greatest gratitude to our Staff members, Librarian, and Non-teaching staff members of Computer Centre, who have helped and supported me throughout. Last but not the Least, I wish to thank my parents and my friends for their individual support and interest who has inspired me and encouraged me to go in my own way, without them I would be unable to complete this project.

ABDUL KADER MAAHIR S - 24619202

P. KISHORE - 24619262

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**i**
	LIST OF FIGURES	**ii**
	LIST OF TABLES	**ii**
	ABBREVIATIONS	**iii**
1	**INTRODUCTION**	**1**
	1.1 OVERVIEW OF THE PROJECT	1
	1.2 SCOPE AND OBJECTIVE	3
2	**LITERATURE REVIEW**	**5**
	2.1 LITERATURE OVERVIEW	5
	2.2 LITERATURE SURVEY	5
3	**SYSTEM DESIGN**	**7**
	3.1 EXISTING SYSTEM	7
	3.2 PROPOSED SYSTEM	7
	3.3 SYSTEM ARCHITECTURE	8
	3.4 HARDWARE REQUIREMENTS	12
	3.5 SOFTWARE REQUIREMENTS	12
4	**IMPLEMENTATION**	**14**
	4.1 METHODOLOGY	14
	4.2 IMPLEMENTATION	17
5	**CONCLUSION AND FUTURE WORK**	**20**
	5.1 CONCLUSION	20
	5.2 FUTURE WORK	21
	5.3 SOURCE CODE	21
	5.4 RESULTS	22
	REFERENCES	**24**

ABSTRACT

Setting a career goal is easy, but being able to identify the skills gap and study the missing skills on every other day is more challenging. This paper presents GrowthAI, a web application that joins them into a whole, instead of treating them as separate processes. A learner sets a goal and the system recommends a roadmap based upon the skills they need to develop. A timed multiple-choice assessment with server-controlled time, shuffled questions and options, and limited attempts measures the current level. Thresholds of fixed scores distinguish beginner, intermediate and advanced levels of skills, and the lower their present level, the higher they appear on the roadmap. Tasks are suggested daily based on available time, recent energy check-ins and skipping history, and every skipped task is recorded with its reason. Both the planner and difficulty recommendation follow explicit rules. A separate logistic-regression module provides an advisory estimate of whether a pending task will be completed. It does not interfere with the plan, and it only returns a value when the task log contains enough completed and skipped examples. The prototype has used React, FastAPI and PostgreSQL, together with bcrypt password hashing, signed JWT tokens and ownership checks. A functional test with the Data Analyst goal showed that the modules work together, but no user study has been performed, and an LLM-based coach and a weekly reflection remain future work.

Keywords: Personal growth assistant, Skill-gap analysis, Adaptive task planning, Logistic regression, Web application

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 3.1	End-to-end workflow of GrowthAI	7
Figure 3.2	Three-layer architecture of GrowthAI	8
Figure 3.3	Level 0 data flow diagram	9
Figure 3.4	Level 1 data flow diagram	10
Figure 3.5	Entity-relationship diagram (main entities)	10
Figure 4.1	Secure assessment flow	15
Figure 4.2	Daily plan generation	16
Figure 4.3	Advisory prediction pipeline	16

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
Table 2.1	Literature survey and relevance to GrowthAI	6
Table 3.1	Existing approach versus GrowthAI	8
Table 3.2	Tables of the GrowthAI database	11
Table 3.3	Features of the prediction model	11
Table 3.4	Hardware requirements	12
Table 3.5	Software requirements	12
Table 3.6	Software modules and files	13
Table 4.1	Development phases	14
Table 4.2	Score thresholds and roadmap priority	15
Table 4.3	Difficulty rule of the daily planner	16
Table 4.4	Functional test scenarios	17
Table 5.1	Module-wise implementation status	22
Table 5.2	Result of one functional assessment (Data Analyst goal)	23

ABBREVIATIONS

ABBREVIATION	FULL FORM
AI	Artificial Intelligence
API	Application Programming Interface
bcrypt	Adaptive password hashing function
DFD	Data Flow Diagram
ER	Entity-Relationship
HMAC	Hash-based Message Authentication Code
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MCQ	Multiple-Choice Question
ML	Machine Learning
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface

CHAPTER 1. INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

Introduction

Almost every learner has a goal, such as becoming a data analyst or cracking a competitive examination. Having a goal is not the same as having a plan. The learner must know which skills the goal needs, how strong he or she is in each skill today, and what to do on a normal day with limited time and energy. Most tools answer only one of these questions, and they answer it in isolation.

GrowthAI is a web-based personal growth and time-management assistant that answers all three questions in one workflow. The learner creates a goal; the system links it to the skills it requires; a timed, secure multiple-choice assessment measures the current level of each skill; the results become skill levels, a skill-gap report and a prioritised roadmap; and a daily planner turns the roadmap into small tasks that fit the time and energy of the day. Every completed or skipped task is recorded, and every skip carries a reason, so that the next plan is built on what actually happened.

Three kinds of intelligence are used, and the report keeps them apart. The skill levels, the roadmap, the daily planner and the difficulty recommendation are **rule-based** and explainable. A **logistic-regression** model trained on the learner's own task history gives an **advisory** estimate of whether a pending task will be completed. A large language model (the Gemini API) is used only to prepare content, such as assessment questions and skill suggestions. An AI Coach and a weekly AI reflection are planned and are not part of the implemented system.

Problem Statement

Learners who work towards a career or examination goal face these problems:

- **Unknown skill gaps:** they do not know which skills a goal needs or how well they perform in each.
- **Static plans:** a fixed timetable ignores the minutes and energy available on a given day, so it is soon abandoned.
- **Lost feedback:** the reason for a skipped task is not recorded, so nobody learns from it.
- **Disconnected tools:** goals, assessment, planning and tracking live in different tools.
- **Weak assessments:** casual online quizzes can be looked up, repeated or shared.

Objectives of the Project

- To provide secure registration and login using bcrypt password hashing and signed JSON Web Tokens (JWT).
- To link every goal to the skills it requires and to measure each skill with a timed, tamper-resistant assessment.
- To convert scores into skill levels, a skill-gap report and a roadmap.
- To generate a rule-based adaptive daily plan from the roadmap, the available minutes, the latest energy check-in and the skip rate.
- To record every task outcome, with a compulsory reason for skips, and to show progress insights.
- To give an advisory logistic-regression prediction of task completion that handles insufficient data honestly.
- To use a language model only for content preparation and to validate its output before use.

Contributions of the Work

- A single workflow that connects goal, skills, assessment, roadmap, daily plan and behaviour feedback.
- A secure assessment design in which timing, ordering and scoring are controlled by the server and the correct answer never reaches the browser.
- An explainable difficulty rule that adapts the daily workload to energy and skip history.
- A leakage-aware advisory prediction module that refuses to predict when the data is insufficient.
- A validated pipeline for AI-assisted question and skill preparation with a model fallback chain.

System Overview

The system has three connected parts that form a feedback loop from goal to plan and back to the plan.

1. **Assessment and gap analysis:** a goal is linked to skills, a timed assessment measures each skill, and fixed thresholds give a level.
2. **Adaptive daily planning:** the planner reads the roadmap, available minutes, latest energy and skip rate and creates one task per skill.
3. **Behaviour tracking and prediction:** outcomes and skip reasons are stored, insights are shown, and an advisory estimate is given once enough history exists.

Advantages of the System

- **Connected workflow:** one application replaces a to-do list, a calendar and a quiz site.
- **Explainable decisions:** every recommendation can be explained with a stated rule.
- **Realistic plans:** the workload follows the learner's minutes and energy.
- **Trustworthy assessment:** the server controls time, order and scoring.
- **Honest use of ML:** the prediction is advisory and says 'insufficient data' when it cannot be trusted.

Applications of the Project

- **Students and job seekers:** structured preparation for a career goal such as data analyst or web developer.
- **Examination aspirants:** subject-wise gap analysis and daily study planning.
- **Self-learners:** a personal planner that adapts to irregular schedules.

1.2 SCOPE AND OBJECTIVE

Purpose of the Study

The purpose of this project is to design, implement and evaluate a goal-to-daily-plan assistant that identifies skill gaps with an objective assessment and plans each day according to the learner's real condition, while keeping every personalisation rule explainable.

General Objective

To build a working web application that connects goal setting, secure skill assessment, gap analysis, adaptive daily planning and behaviour tracking, and that offers an honest advisory prediction of task completion.

Specific Objectives

- To develop the backend services for authentication, goals, skills, assessment, tasks and insights.
- To design a database that stores goals, skills, attempts, answers, tasks and logs.
- To implement the difficulty rule and the daily plan generator.
- To train and expose the advisory logistic-regression model with data-sufficiency checks.

- To build a calm, easy-to-use React interface for all modules.

Scope of the Project

A. Functional scope. Registration and login; profile; goals and goal-skill mapping; secure timed assessment; skill levels, gap report and roadmap; rule-based daily planner; task and skip logging; energy check-ins; progress insights; advisory prediction.

B. Technical scope. React 19 and Vite on the front end; Python, FastAPI and Uvicorn on the back end; PostgreSQL on Neon; scikit-learn for the advisory model; the Gemini API for content preparation only.

C. Application scope. Individual learners preparing for a career or examination goal. An LLM-based AI Coach, a weekly AI reflection and a learned planning policy are planned and are not claimed as completed. Classroom management and medical or financial advice are outside the scope.

Importance of the Project

Learners lose time when they study what they already know and avoid what they find difficult. A tool that measures skills objectively, plans realistically and treats skipping as feedback helps learners spend their limited time where it matters most.

Challenges and Limitations

- The question bank is small and AI-generated questions are not reviewed by a subject expert.
- The prediction model needs several logged outcomes before it can be used.
- The free tier of the AI service allows only a small number of requests per day for each model.
- No user study has been carried out yet.

CHAPTER 2. LITERATURE REVIEW

2.1 LITERATURE OVERVIEW

GrowthAI sits at the meeting point of several research areas: goal setting and self-regulated learning, personalised and adaptive learning, behaviour change and procrastination, prediction on small personal datasets, and secure web application design. This chapter reviews representative work in each area and explains how it shaped the design. The review is not exhaustive; its purpose is to justify design decisions.

2.2 LITERATURE SURVEY

Goal setting and self-regulation

Locke and Latham showed that specific, appropriately challenging goals lead to better performance than vague goals [3]. This supports turning a broad goal into concrete skills and daily tasks. Steel's meta-analysis describes procrastination as a failure of self-regulation linked to task aversiveness and low confidence [2], which motivates sizing tasks to the learner's energy and storing the reason for every skip.

Personalised and adaptive learning

Bloom reported that one-to-one tutoring greatly outperforms conventional teaching and asked for methods that approach it at scale [1]. A meta-analysis by Kulik and Fletcher found that intelligent tutoring systems generally improve learning [10]. Knowledge tracing estimates what a learner knows from the sequence of answers, from Bayesian knowledge tracing [4] to recurrent neural networks [5]. GrowthAI does not implement knowledge tracing; it adopts the idea that the current level should drive the next step and realises it with simple score thresholds.

Behaviour change, prediction and security

Fogg's model states that a behaviour occurs when motivation, ability and a prompt coincide [11]; GrowthAI therefore reduces task size when energy is low or skips are frequent. Logistic regression is an interpretable method for binary outcomes that works with small data [7] and is available in scikit-learn [6]. Passwords should be stored with a slow salted hash such as bcrypt [8], and stateless sessions can use JSON Web Tokens [9]. The REST style [12] underlies the interface between browser and server.

Table 2.1: Literature survey and relevance to GrowthAI

Ref.	Work	Relevance to GrowthAI
[1]	Bloom (1984), the 2 sigma problem	Motivates level-aware individual guidance
[2]	Steel (2007), procrastination	Record skip reasons; size tasks to the learner
[3]	Locke and Latham (2002), goal setting	Goals broken into concrete skills and tasks
[4]	Corbett and Anderson (1994), knowledge tracing	Current level drives the next step
[5]	Piech et al. (2015), deep knowledge tracing	Learned alternative; future work
[6]	Pedregosa et al. (2011), scikit-learn	Library for the advisory prediction
[7]	Hosmer et al. (2013), logistic regression	Interpretable prediction model
[8]	Provos and Mazieres (1999), bcrypt	Password hashing
[9]	Jones et al. (2015), JWT, RFC 7519	Token-based sessions
[10]	Kulik and Fletcher (2016), tutoring systems	Evidence for adaptive systems
[11]	Fogg (2009), behaviour model	Task sizing and daily prompts
[12]	Fielding (2000), REST	Client-server API style

Research Gaps Identified

- Planner applications record tasks but do not know which skills a goal needs.
- Course platforms deliver content but do not adapt each day's workload to the learner's energy.
- Online quizzes measure knowledge but are not linked to a plan and are easy to cheat.
- Prediction features are rarely explained or gated by the amount of data available.

Summary of Literature Findings

The literature supports concrete goals [3], level-aware guidance [1], [4], behaviour-aware task sizing [2], [11], an interpretable prediction model [6], [7] and standard security practice [8], [9]. GrowthAI combines these ideas in one application and validates every AI-generated question before it is stored.

CHAPTER 3. SYSTEM DESIGN

3.1 EXISTING SYSTEM

Overview

Learners combine general-purpose tools: a to-do application for tasks, a calendar for time, courses or videos for content and an occasional online quiz. The learner guesses the skills a goal needs, estimates his or her own level, writes a fixed timetable and ticks tasks off. When the plan fails it is rewritten by hand.

Limitations of the Existing Systems

- No link between a goal and the skills it requires.
- Self-estimated levels, so learners study what they already know.
- A static workload that ignores energy and available time.
- No reason recorded for skipped tasks, so history is lost.
- Assessments that can be repeated or looked up, so scores are unreliable.

3.2 PROPOSED SYSTEM

Overview

GrowthAI replaces these tools with one feedback loop (Figure 3.1): goal, skills, secure assessment, levels and gaps, roadmap, daily plan, task outcomes, insights and advisory prediction.

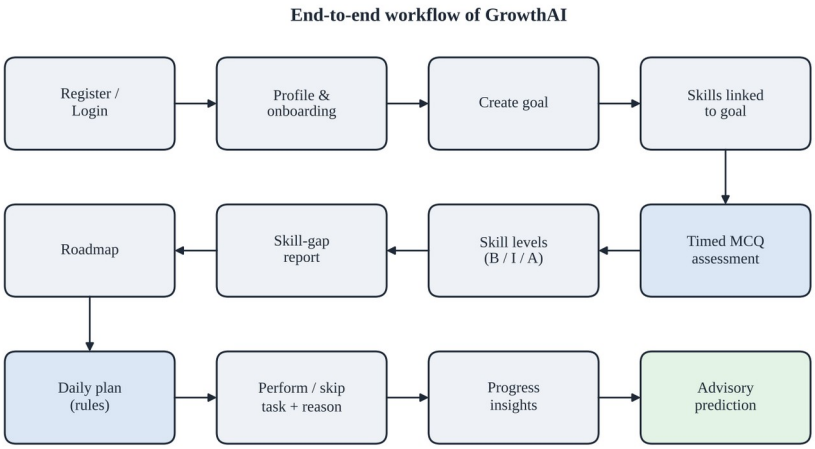


Figure 3.1: End-to-end workflow of GrowthAI

Features of the Proposed System

- **Goal-to-skill mapping** with an importance (high, medium, low) for each skill.

- **Secure timed assessment** with server clock, shuffled order, hidden answers and an attempt limit.
- **Skill levels, gap report and roadmap** from fixed score thresholds.
- **Adaptive daily plan** sized by energy, skip rate and available minutes.
- **Skip with reason**, summarised in the insights.
- **Advisory prediction** of task completion with data-sufficiency checks.

Table 3.1: Existing approach versus GrowthAI

Aspect	Existing approach	GrowthAI
Skill identification	Guessed by the learner	Goal linked to skills with importance
Level measurement	Self-estimation	Timed, secure MCQ assessment
Daily workload	Fixed timetable	Sized by minutes, energy and skip rate
Skipped tasks	Only marked overdue	Reason required and summarised
Prediction	None	Advisory logistic regression
Assessment integrity	Can be looked up	Server clock, hidden answers, attempt limit

3.3 SYSTEM ARCHITECTURE

Architectural Overview

GrowthAI has three layers (Figure 3.2). The layers communicate only through defined interfaces, so each can be tested and replaced independently.

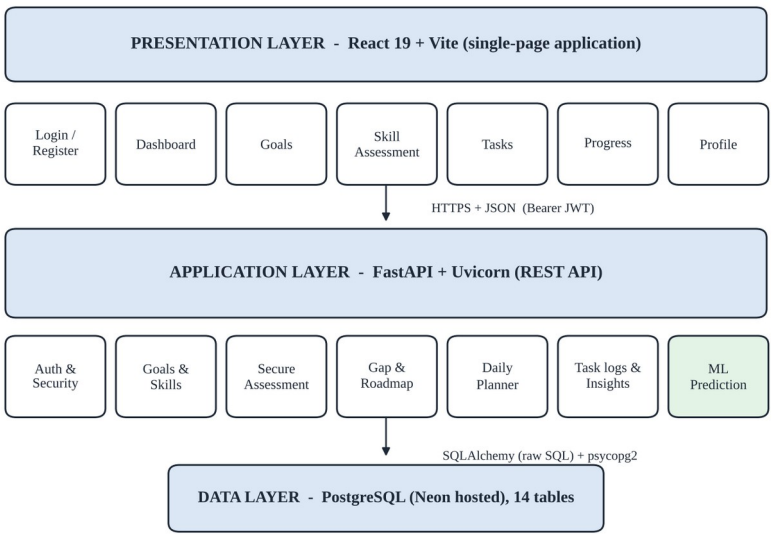


Figure 3.2: Three-layer architecture of GrowthAI

Users and Input Layer

The user interface is a React 19 single-page application built with Vite. The active page (login, dashboard, goals, assessment, tasks, progress, profile) is selected by a state variable. The browser sends JSON over HTTPS and attaches the JWT as a bearer token. The assessment page runs the countdown timer, counts tab switches for information and disables copy, cut and the context menu.

Application Layer

The server is a FastAPI service run by Uvicorn. It validates requests with Pydantic schemas and checks on every protected route that the record belongs to the authenticated user. Heavy libraries such as scikit-learn are imported only when the prediction endpoint is called, so the server starts quickly.

Data Layer

PostgreSQL, hosted on Neon, stores all data in 14 tables. The server uses SQLAlchemy with parameterised raw SQL and the psycopg2 driver. Because a hosted database may close idle connections, the server opens short connections and never keeps one open while waiting for a slow external call.

Data Flow Diagrams

In the Level 0 diagram (Figure 3.3) the learner is the only external entity. Level 1 (Figure 3.4) divides the system into seven processes and four data stores.

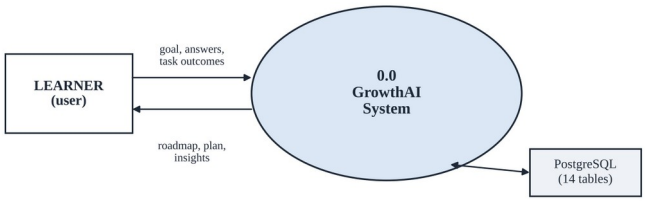


Figure 3.3: Level 0 data flow diagram

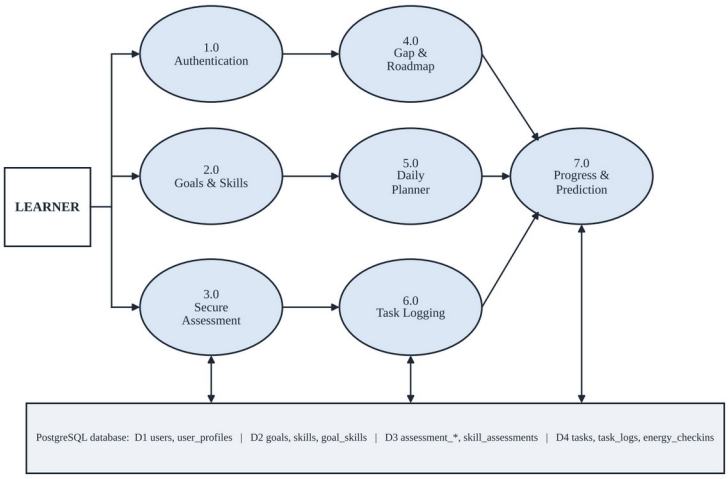


Figure 3.4: Level 1 data flow diagram

Database Design

A user has one profile and many goals. A goal is linked to many skills through goal_skills. Assessment data is split into attempts, answers and per-skill results so that an attempt can be resumed and an answer stored only once (Figure 3.5, Table 3.2).

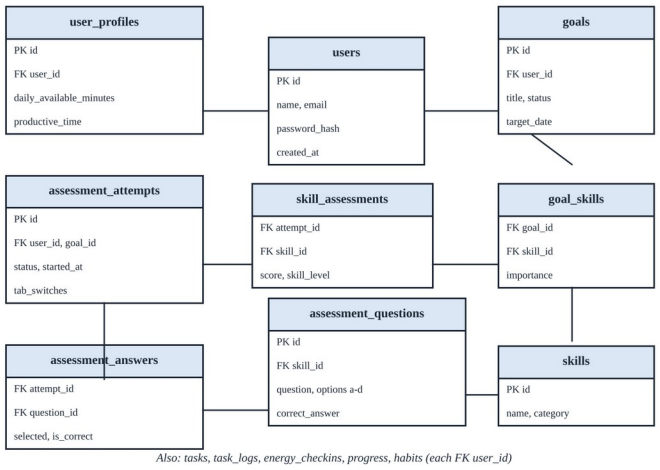


Figure 3.5: Entity-relationship diagram (main entities)

Table 3.2: Tables of the GrowthAI database

Table	Purpose
users, user_profiles	Account with bcrypt hash; daily minutes, productive time, sleep, procrastination level
goals, skills, goal_skills	Goals; master skill list; goal-to-skill link with importance
assessment_questions	MCQs with four options and the correct letter
assessment_attempts, assessment_answers	Attempt status, start time, tab switches; one answer per attempt and question
skill_assessments	Score and level per skill and attempt
tasks, task_logs	Planned tasks; completion and skip events with reason
energy_checkins, progress, habits	Energy reports; progress measures; habit records

Architectural Flow

1. **Input:** the learner supplies a goal, answers and task outcomes through the web interface.
2. **Authentication:** the server verifies the token and loads the user.
3. **Assessment:** the server builds a shuffled presentation, keeps the clock, scores answers and stores levels.
4. **Roadmap and plan:** levels are ordered into a roadmap; the planner creates the day's tasks.
5. **Logging:** outcomes and reasons are stored.
6. **Insights and prediction:** summaries are computed and, when data allows, an advisory estimate is returned.

Feature Representation for Prediction

The advisory model uses nine features for each logged outcome (Table 3.3). All history-based features are computed only from outcomes that happened before the task, which avoids leakage.

Table 3.3: Features of the prediction model

No.	Feature	Meaning
1	Duration	Planned minutes of the task
2	Duration missing	1 if no duration was set
3	Available minutes	Learner's daily available minutes

No.	Feature	Meaning
4	Energy value	Latest energy check-in before the outcome (1 low, 2 medium, 3 high)
5	Completion rate	Share of earlier tasks completed
6	Skip rate	Share of earlier tasks skipped
7	Recent skips	Skips among the last five outcomes
8	Difficulty	Easy 1, medium 2, hard 3
9	Productive-time match	1 if the scheduled time falls in the learner's productive period

3.4 HARDWARE REQUIREMENTS

Table 3.4: Hardware requirements

Component	Development machine	Client (learner)
Processor	Intel Core i5 or equivalent	Any modern dual-core processor
Memory	8 GB RAM	4 GB RAM
Storage	5 GB free space	Not required
Network	Broadband internet connection	Stable internet connection
Display	Standard monitor	Screen of 360 px width or more

3.5 SOFTWARE REQUIREMENTS

Software Tools

Table 3.5: Software requirements

Item	Specification
Operating system	Windows 10 or 11 (development); any OS with a modern browser (client)
Backend	Python 3.11 or later, FastAPI, Uvicorn, SQLAlchemy, psycopg2, bcrypt, Pydantic, scikit-learn
Frontend	React 19, Vite, Node.js; Chrome, Edge or Firefox
Database and API	PostgreSQL (Neon); Google Gemini API (optional)
Tools	Visual Studio Code, PowerShell, Git

Software Modules

Table 3.6: Software modules and files

File	Purpose
main.py	FastAPI routes: accounts, goals, assessment, tasks, insights, prediction
security.py	bcrypt hashing, password policy, JWT creation and verification
schemas.py	Pydantic request and response models
database.py	Database engine and environment loading
schema.sql, seed.sql	Table definitions and starter skills and questions
ai_client.py	Standard-library client for the AI service with model fallback
generate_questions.py	Question generation with validation and second-opinion check
ai_goal_skills.py	AI skill mapping and background question preparation
App.jsx, pages, components	React application shell, pages and shared components

CHAPTER 4. IMPLEMENTATION

4.1 METHODOLOGY

Development Approach

The project followed an incremental approach. The core of the system was built first, and each later increment was tested on a running copy before the next was started (Table 4.1).

Table 4.1: Development phases

Phase	Work carried out
1. Requirements	Goal, skill, assessment, planning and tracking needs; scope and limits
2. Design	Architecture, data flow diagrams, 14-table schema, API list
3. Core backend	Authentication, goals, skills, roadmap, tasks, task logs
4. Secure assessment	Server clock, HMAC shuffle, idempotent answers, attempt limit
5. Planner and ML	Difficulty rule, daily plan, advisory logistic regression
6. Frontend	React pages for each module and the assessment timer
7. AI content	Gemini client, question generation with validation, skill mapping
8. Testing	Functional walk-through with the Data Analyst goal

Assessment Design

Each question allows 120 seconds plus a 30-second grace period, and the deadline is computed on the server clock. The order of options is derived from an HMAC keyed with the server secret and recomputed on every request, so no shuffle is stored. The learner sees shuffled letters and the server translates them back to the stored letters when scoring. Figure 4.1 shows the flow.

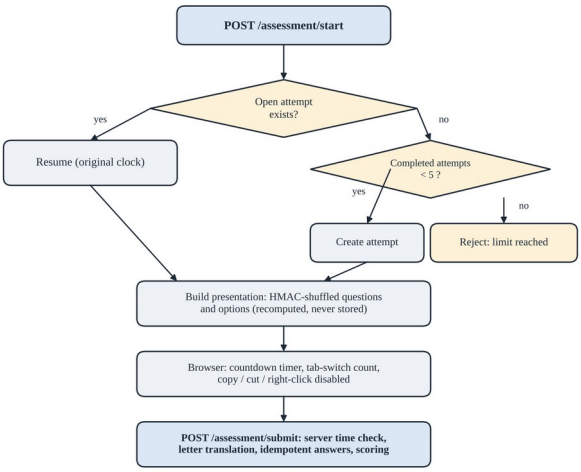


Figure 4.1: Secure assessment flow

Skill Levels and Roadmap

A score below 40 is beginner, from 40 up to but not including 70 is intermediate, and 70 or above is advanced (Table 4.2). Beginner and intermediate skills are the learner's gaps and share one priority. The roadmap lists gap skills first, then skills not yet assessed, then advanced skills; ties are broken by importance (high, medium, low).

Table 4.2: Score thresholds and roadmap priority

Score	Level	Roadmap position
Below 40	Beginner	First group (gap skill)
40 to below 70	Intermediate	First group (gap skill)
Not assessed	Unknown	Second group
70 or above	Advanced	Last group

Adaptive Daily Planner

Figure 4.2 shows plan generation. Candidates are ordered with not-assessed skills first, then gap skills, then strong skills, ties by importance, with one task per skill. The task size follows Table 4.3, using the latest energy check-in and the skip rate (skipped and postponed tasks divided by all logged tasks). Tasks are added while they fit the daily available minutes (default 60). The same rule is available at GET /tasks/adaptive-recommendation.

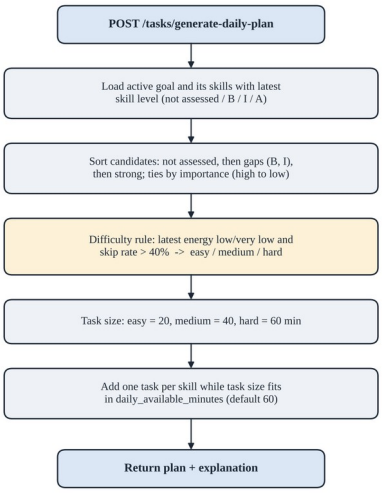


Figure 4.2: Daily plan generation

Table 4.3: Difficulty rule of the daily planner

Energy low or very low	Skip rate above 40%	Difficulty	Task length
Yes	Yes	Easy	20 minutes
Yes	No	Medium	40 minutes
No	Yes	Medium	40 minutes
No	No	Hard	60 minutes

Advisory Prediction

A logistic-regression model with StandardScaler is trained on the learner's own outcomes (Figure 4.3). A prediction needs at least six outcomes with both completed and skipped examples. Hold-out metrics are reported only with at least ten outcomes and at least two in each class. Otherwise the module reports insufficient data. The result never changes the plan.

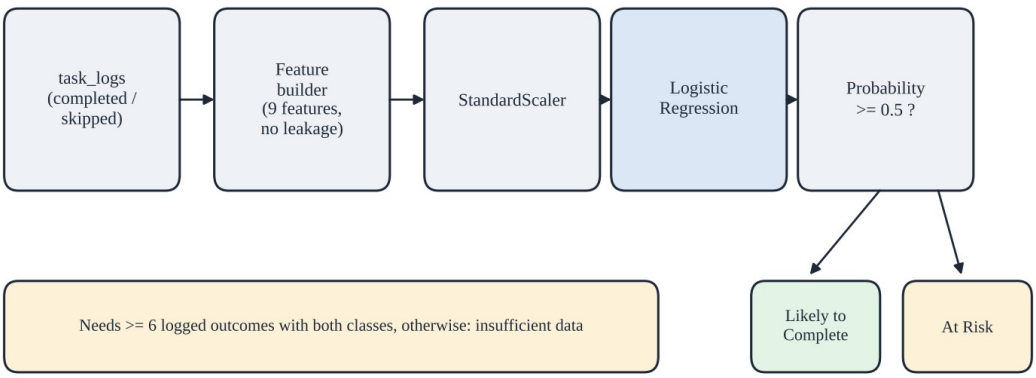


Figure 4.3: Advisory prediction pipeline

AI-Assisted Content Preparation

A small Gemini client written with the Python standard library suggests skills for a new goal and generates assessment questions. A generated question must have four distinct options, a valid answer, no 'all/none of the above' and no duplicate. A second independent model call then answers it without seeing the key, and only questions where both agree are saved, with option positions shuffled. If a model reaches its quota the client moves to the next model in a fallback chain.

Testing and Validation

Testing was functional and was carried out by the developers on a running system. No user study and no model-accuracy evaluation was performed. Table 4.4 lists the scenarios and their basis.

Table 4.4: Functional test scenarios

No.	Scenario	Expected result	Status
1	Register with a weak password	Rejected with a message	Rule in code
2	Register, log in, open a protected page	Token issued; page loads	Functional test
3	Create the goal 'Become a Data Analyst'	Goal saved; skills linked	Functional test
4	Start, answer and complete an assessment	Levels and gap report shown	Functional test
5	Resume an open attempt	Original clock and questions kept	Rule in code
6	Generate roadmap and daily plan	Gap skills first; plan fits minutes	Functional test
7	Skip a task without a reason	Rejected; reason required	Rule in code
8	Prediction with little history	'Insufficient data' message	Rule in code
9	Goal with no template (AI skills)	Skills and questions prepared in background	Implemented; live check pending

4.2 IMPLEMENTATION

4.2.1 Authentication Module

Technology used: Python, bcrypt, HS256 JWT. **Files:** security.py, main.py. A password needs at least eight characters, a letter and a digit, and at most 72 bytes

(the bcrypt limit). It is hashed with bcrypt before storage. Login issues a signed token valid for 1440 minutes, and every protected route decodes the token and loads the user.

4.2.2 Goals and Skills Module

Technology used: FastAPI, SQLAlchemy, Gemini API. **Files:** main.py, ai_goal_skills.py. When a goal is created, a stored template with the same lowercase title is used if one exists. For a new topic the server asks the model for four to six core skills in a background thread, so goal creation never waits for the model. Names are cleaned and de-duplicated; if the model is unavailable the goal is still created, without skills.

4.2.3 Secure Assessment Module

Technology used: HMAC-SHA256, PostgreSQL unique constraints. **File:** main.py. The module implements the server clock, the shuffle, the letter translation and the attempt limit of at most five completed or expired attempts per goal. A unique key on attempt and question makes answers idempotent, so repeating a request cannot change a score. Tab switches are counted for information only.

4.2.4 Gap and Roadmap Module

Technology used: SQL, Python. **File:** main.py (/goals/{id}/skill-gaps, /goals/{id}/roadmap). The module reads the latest skill level of each skill of the goal and returns the gap list and the ordered roadmap described in Section 4.1.

4.2.5 Daily Planner Module

Technology used: Python, SQL. **File:** main.py (/tasks/generate-daily-plan). The function _recommended_difficulty reads the latest energy check-in and the skip rate and returns easy, medium or hard together with a plain-language explanation that is shown to the learner.

4.2.6 Task Logging and Insights Module

Technology used: FastAPI, PostgreSQL. **File:** main.py (/task-logs, /task-logs/procrastination-summary, /progress-summary). A task is marked completed, skipped or postponed. A reason is compulsory for a skip. The insights show the completion rate, the skip rate and the most common skip reasons.

4.2.7 Prediction Module

Technology used: scikit-learn (StandardScaler, LogisticRegression). **File:** main.py (/tasks/prediction). Training examples come only from the same user and are ordered by outcome time. The module imports scikit-learn lazily and returns an advisory probability only when the data rules are met.

4.2.8 AI Content Module

Technology used: Python standard library, Gemini API. **Files:** ai_client.py, generate_questions.py. The client tries the configured model and then fallback models; a model that returns a quota error is skipped for an hour. Preparation runs in background threads with short database connections so that the hosted database does not close an idle connection during a slow AI call.

4.2.9 Front End

Technology used: React 19, Vite, CSS. **Files:** App.jsx, pages/*.jsx, components/*.jsx. A single-page application switches between pages with the activePage state. Pages exist for authentication, dashboard, goals, assessment, tasks, progress and profile. The assessment page runs the timer and disables copying.

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

Purpose and Achievements

GrowthAI joins goal setting, objective assessment, gap analysis, adaptive daily planning and behaviour tracking in one working web application. A learner can move from a goal to a skill-gap report, a roadmap and a daily plan, and the outcomes of each day feed the next plan.

Significance of the System

The system treats a skipped task as information instead of failure and keeps every personalisation rule explainable. This makes it suitable for learners who want guidance they can understand and question.

Technical Highlights

- A server-controlled secure assessment with HMAC shuffling and hidden answers.
- An explainable difficulty rule shared by the plan generator and a separate endpoint.
- A leakage-aware advisory model that declines to predict on thin data.
- Background AI preparation with validation, second-opinion checks and model fallback.

Evaluation and Implementation Status

A functional test with the Data Analyst goal showed that the modules work together. No user study and no model-accuracy evaluation has been done. The AI Coach and the weekly reflection are planned and are not implemented.

Challenges Faced

- The hosted database closes idle connections, so the server uses short connections and never holds one across an AI call.
- The free tier of the AI service limits each model to a small number of requests a day, so a fallback chain was added.
- Goal creation once waited for the model; skill mapping was moved to background threads.
- Goals with titles that did not match a stored template had no skills, which led to AI skill mapping.

Limitations

- No user study has been done, so the benefit to learners is not measured.
- Question banks are small; AI-generated questions are validated but not reviewed by a subject expert.
- The prediction model needs several logged outcomes and uses only nine features.

5.2 FUTURE WORK

- **AI Coach and weekly reflection:** an LLM-based conversational coach with a per-user daily limit.
- **Learned planning policy:** replace some fixed rules and evaluate them against the current rules.
- **User study:** test with real learners and expert-reviewed question banks.
- **Notifications and calendar integration:** reminders at the learner's productive time.
- **Security and production readiness:** rate limiting, audit logs and deployment hardening.

5.3 SOURCE CODE

This section shows selected excerpts from the backend.

Skill Level Thresholds (main.py)

```
if score_percent < 40:
    skill_level = "beginner"
elif score_percent < 70:
    skill_level = "intermediate"
else:
    skill_level = "advanced"
```

Difficulty Rule (main.py)

```
low_energy = latest_energy is not None and \
    latest_energy.lower() in ("low", "very low")
high_skip = skip_rate > 40
if low_energy and high_skip:   -> "easy"
elif low_energy or high_skip:  -> "medium"
else:                          -> "hard"
```

Server-Side Option Shuffle (main.py)

```
def _attempt_hmac(attempt_id, purpose, *parts):
    msg = ":".join([purpose, str(attempt_id), *map(str, parts)])
    return hmac.new(get_jwt_secret(), msg.encode(),
                    hashlib.sha256).hexdigest()

def _shuffle_options(attempt_id, question_id, options):
    return sorted(options.items(), key=lambda it:
        (_attempt_hmac(attempt_id, "opts", question_id, it[0]), it[0]))
```

Model Fallback (ai_client.py)

```
for model in _model_chain(prefer):
    if _exhausted_until.get(model, 0) > time.time():
        continue            # skip a model that hit its quota
    ...
    if exc.code in (429, 404):
        _exhausted_until[model] = time.time() + COOLDOWN_SECONDS
        break               # try the next model straight away
```

5.4 RESULTS

Implementation Status and Prototype

The implemented system covers the complete path from goal to daily plan and feedback. Table 5.1 summarises what was delivered.

Table 5.1: Module-wise implementation status

Module	Status
Secure registration, login, profile	Implemented
Goals and skill mapping (template and AI)	Implemented
Timed secure assessment with attempt limit	Implemented
Skill levels, gap report, roadmap	Implemented
Rule-based adaptive daily planner	Implemented
Task and skip logging, insights	Implemented
Advisory logistic-regression prediction	Implemented (advisory)
AI Coach and weekly reflection	Planned

Evaluation Result

One developer completed a seven-question assessment for the Data Analyst goal (Table 5.2). It shows that scoring, levels and the roadmap work together; it is a single attempt and not a measure of learning benefit.

Table 5.2: Result of one functional assessment (Data Analyst goal)

Skill	Correct / asked	Score	Level
Python	1 / 2	50%	Intermediate
SQL	0 / 2	0%	Beginner
Statistics	0 / 1	0%	Beginner
Excel	1 / 1	100%	Advanced
Data Visualization	0 / 1	0%	Beginner
Overall	2 / 7	28.57%	-

Discussion

The roadmap placed the four gap skills ahead of Excel, which was advanced, as the rule requires; Python (intermediate) and the beginner skills share one priority and were ordered by importance. The result is from seven questions only, so it should be read as a demonstration of the pipeline.

Limitations of the Results

There is no user study, no measure of learning gain, and no accuracy figure for the prediction model. These are listed as future work.

REFERENCES

[1] B. S. Bloom, "The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring," Educational Researcher, vol. 13, no. 6, pp. 4-16, 1984.

[2] P. Steel, "The nature of procrastination: A meta-analytic and theoretical review of quintessential self-regulatory failure," Psychological Bulletin, vol. 133, no. 1, pp. 65-94, 2007.

[3] E. A. Locke and G. P. Latham, "Building a practically useful theory of goal setting and task motivation: A 35-year odyssey," American Psychologist, vol. 57, no. 9, pp. 705-717, 2002.

[4] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," User Modeling and User-Adapted Interaction, vol. 4, no. 4, pp. 253-278, 1994.

[5] C. Piech et al., "Deep knowledge tracing," in Advances in Neural Information Processing Systems 28, 2015, pp. 505-513.

[6] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[7] D. W. Hosmer, S. Lemeshow and R. X. Sturdivant, Applied Logistic Regression, 3rd ed. Hoboken, NJ: Wiley, 2013.

[8] N. Provos and D. Mazieres, "A future-adaptable password scheme," in Proc. USENIX Annual Technical Conference, FREENIX Track, 1999, pp. 81-91.

[9] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015.

[10] J. A. Kulik and J. D. Fletcher, "Effectiveness of intelligent tutoring systems: A meta-analytic review," Review of Educational Research, vol. 86, no. 1, pp. 42-78, 2016.

[11] B. J. Fogg, "A behavior model for persuasive design," in Proc. 4th Int. Conf. Persuasive Technology, 2009, Art. 40.

[12] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. California, Irvine, 2000.